

Security Assessment Report

Aplicación: Docker Manager v1.0.0 | **Fecha:** 15 de junio de 2026 | **Autor:** JC | **Tipo:** Auditoría de código estático + análisis de dependencias

A

Calificación global

0

Vulnerabilidades críticas

0

Vulnerabilidades altas

2

Issues corregidos

3

Recomendaciones menores

1. Resumen Ejecutivo

Docker Manager es una aplicación Electron de escritorio para gestión y despliegue de proyectos Docker. La evaluación de seguridad cubre el análisis estático del código fuente (main.js, preload.js, renderer/app.js, renderer/index.html), la revisión de dependencias npm y la configuración de hardening de Electron.

La aplicación demuestra **prácticas de seguridad maduras**: uso de `spawn()` con arrays de argumentos, validación estricta de inputs, y todas las opciones de hardening recomendadas por la documentación de Electron. Se identificaron y corrigieron 2 issues menores durante esta auditoría.

Resultado: **APTO PARA PRODUCCIÓN** tras aplicar las correcciones documentadas en la sección 4.

2. Alcance de la Auditoría

Componente	Archivo	LOC	Estado
Proceso principal	main.js	~2,500	Revisado
Preload bridge	preload.js	~122	Revisado
Renderer UI	renderer/app.js	~4,200	Revisado
HTML / CSP	renderer/index.html	~400	Revisado
Dependencias npm	package.json / package-lock.json	—	npm audit: 0 CVEs
Instalador	assets/installer.nsh	~50	Revisado

3. Análisis de Dependencias

Resultado de npm audit : **0 vulnerabilidades**

Paquete	Versión	Tipo	Propósito	Estado
electron	^42.4.0	devDependency	Framework desktop	Sin CVEs

electron-builder	^26.15.3	devDependency	Empaquetado e instaladores	Sin CVEs
------------------	----------	---------------	----------------------------	----------

La aplicación usa únicamente módulos nativos de Node.js en tiempo de ejecución (child_process, fs, path, os, https, util). No existen dependencias de producción de terceros, eliminando prácticamente toda la superficie de ataque de la cadena de suministro.

4. Hallazgos y Correcciones

4.1 Issues Corregidos en Esta Auditoría

#	Severidad	Descripción	Archivo / Línea	Estado
SEC-01	LOW	DevTools accesible en producción via menú El ítem "DevTools" del menú Vista y su atajo <code>Ctrl+Shift+I</code> eran visibles en builds de producción. Aunque <code>devTools: IS_DEV</code> evitaba que se abrieran, el ítem de menú podía confundir usuarios.	<code>main.js:191</code>	CORREGIDO
SEC-02	MEDIUM	Parámetro <code>version</code> sin validar en <code>app:updatePackage</code> El campo <code>version</code> recibido del renderer se concatenaba directamente como <code>name@version</code> sin sanitización, permitiendo potencialmente inyectar flags de npm (ej. <code>@latest --global</code>).	<code>main.js:1068</code>	CORREGIDO

4.2 Recomendaciones Menores (Sin Riesgo Inmediato)

#	Severidad	Descripción	Esfuerzo
REC-01	INFO	CSP con <code>'unsafe-inline'</code> para scripts La cabecera Content-Security-Policy permite scripts inline (<code>script-src 'self' 'unsafe-inline'</code>). En una aplicación Electron cargando solo archivos locales el riesgo real es muy bajo, pero migrar los inline scripts a archivos externos permitiría eliminar esta excepción.	Alto
REC-02	INFO	Firma de código del instalador no configurada El campo <code>publish: null</code> y la ausencia de certificados de firma implica que Windows Defender SmartScreen puede mostrar advertencias al instalar. Se recomienda adquirir un certificado EV Code Signing para builds de distribución pública.	Medio
REC-03	INFO	PowerShell con <code>-ExecutionPolicy Bypass</code> La función <code>psRun()</code> invoca PowerShell con <code>-ExecutionPolicy Bypass</code> para operaciones WSL. Es comportamiento intencional y necesario, pero se documenta para que revisiones futuras no lo marquen como error.	—

5. Fortalezas de Seguridad Identificadas

Área	Implementación	Beneficio
Electron hardening	<code>contextIsolation: true</code> , <code>nodeIntegration: false</code> , <code>sandbox: true</code> , <code>webSecurity: true</code> , <code>allowRunningInsecureContent: false</code> , <code>navigateOnDragDrop: false</code>	Aísla el renderer; impide acceso directo a Node.js desde la UI
Ejecución de procesos	<code>spawn('docker', argsArray)</code> — sin shell, sin concatenación de strings	Inmune a inyección de shell vía parámetros del usuario

Validación de inputs	Regex <code>SHELL_INJECT_RE</code> , <code>PATH_INVALID_RE</code> ; whitelist de acciones (<code>ALLOWED_CONTAINER_ACTIONS</code>), providers (<code>ALLOWED_CLOUD_PROVIDERS</code>) y prefijos de comando	Bloquea caracteres de inyección antes de llegar a cualquier proceso
Credenciales	Contraseñas enviadas vía stdin del proceso hijo; enmascaradas en logs	No aparecen en lista de procesos del SO ni en logs
Navegación	<code>will-navigate</code> bloqueado; <code>setWindowOpenHandler</code> deniega nuevas ventanas internas	Previene redirecciones a URLs externas maliciosas
Timeouts	Todos los procesos Docker tienen timeout configurable (default 30 s)	Previene bloqueos por procesos zombis o conexiones remotas colgadas
Superficie de ataque	0 dependencias npm de producción; solo módulos nativos de Node.js	Elimina el riesgo de supply-chain attacks vía paquetes de terceros
HTTPS	Todas las llamadas a registries cloud usan <code>https.get()</code> con timeout de 12 s	Previene ataques MITM en operaciones de autenticación con cloud
Monitoreo de deps	Chequeo automático de <code>npm outdated</code> y <code>npm audit</code> al arrancar, con banner de notificación en UI	El usuario es alertado inmediatamente si aparecen vulnerabilidades en las dependencias

6. Checklist OWASP Desktop App Security

Control OWASP	Estado	Notas
DA-1: Hardening de plataforma	PASS	contextIsolation, sandbox, webSecurity activos
DA-2: Validación de inputs	PASS	sanitizeArg() + regex en todos los inputs IPC
DA-3: Gestión de credenciales	PASS	stdin, no args de línea de comando
DA-4: Criptografía	PASS	HTTPS para todas las llamadas a APIs de registro
DA-5: Comunicación segura	PASS	StrictHostKeyChecking en SSH/SCP
DA-6: Gestión de sesiones	PASS	Tokens no persistidos; se usa docker login del SO
DA-7: Control de acceso	PASS	asInvoker; no requiere privilegios elevados
DA-8: Protección de datos	PASS	Sin almacenamiento local de secretos
DA-9: Logging y auditoría	PARTIAL	Logs de UI disponibles; sin log persistente de auditoría
DA-10: Gestión de errores	PASS	Errores capturados y presentados al usuario sin stack traces
DA-11: Componentes de terceros	PASS	0 deps de producción; npm audit limpio
DA-12: Firma de código	PENDING	Recomendado para distribución pública (ver REC-02)

7. Vectores de Amenaza Evaluados

7.1 Inyección de Comandos (Shell Injection)

Estado: MITIGADO

Todos los comandos Docker se ejecutan con `spawn(binary, argsArray)`. Los argumentos nunca pasan por un shell; la interpolación de strings con input del usuario no ocurre en ningún path de ejecución. El regex `SHELL_INJECT_RE = /[;&|`$<>(){}!\\"/` previene que caracteres peligrosos lleguen siquiera al proceso.

7.2 Cross-Site Scripting (XSS)

Estado: MITIGADO

La función `escHtml()` se aplica consistentemente antes de insertar datos externos en el DOM. El contexto del renderer está aislado (`contextIsolation: true`). Los accesos directos al DOM con `innerHTML` de datos confiables (templates internos) son la excepción documentada.

7.3 Prototype Pollution / Dependency Confusion

Estado: NO APLICA

Al no tener dependencias de producción de terceros, la superficie de ataque de supply-chain es mínima (limitada a Electron y electron-builder, ambas con 0 CVEs activos).

7.4 Escalada de Privilegios

Estado: MITIGADO

El instalador NSIS usa `requestedExecutionLevel: asInvoker`. La aplicación no solicita UAC ni privilegios elevados. Las operaciones Docker requieren que el usuario sea miembro del grupo `docker-users`, lo cual es responsabilidad de Docker Desktop.

8. Plan de Acción

Item	Prioridad	Plazo sugerido	Responsable
SEC-01: DevTools en producción	COMPLETADO	—	JC
SEC-02: Validación de versión en <code>updatePackage</code>	COMPLETADO	—	JC
REC-01: Eliminar <code>unsafe-inline</code> del CSP	BAJO	v1.1	JC
REC-02: Firma de código (Code Signing)	MEDIO	Antes de distribución pública	JC
REC-03: <code>psRun ExecutionPolicy Bypass</code> (documentado)	INFO	—	—
Monitoreo continuo de deps (<code>npm audit</code> automático)	COMPLETADO	—	JC

9. Herramientas y Metodología

- **Análisis estático manual:** revisión completa de `main.js`, `preload.js`, `renderer/app.js`, `index.html`
- **npm audit** v10 — escaneo de CVEs en dependencias
- **npm outdated** — detección de versiones desactualizadas
- **OWASP Desktop App Security Cheat Sheet** — referencia de controles
- **Electron Security Checklist** (docs.electronjs.org/tutorial/security)